Grammar-Based Fuzzing and Differential Testing of Small Hugging Face Chat Models: What It Finds and Where Its Oracles Fail

[Author 1][1], [Author 2][1], [Author 3][1], [Author 4][1]

[1] [Department – University]
[City – State – Brazil]

[{author1,author2,author3,author4}@university.br]

Abstract. *Chat models distributed through the Hugging Face Hub are increasingly embedded in software, yet they are hard to test: their input is natural language, they rarely crash, and most prompts have no single correct answer. We investigate whether grammar-based fuzzing combined with differential testing across models and metamorphic testing across paraphrases is an effective way to find functional failures in such models, and where its oracles fail. Our tool,* `chatfuzz`*, derives prompts from a context-free grammar whose derivations carry their own expected answer (arithmetic, letter counting, string reversal, JSON output, verbatim echo of hard Unicode text) and re-derives each test as paraphrases. We ran 170 prompts on three small open chat models (Qwen2.5-0.5B-Instruct, SmolLM2-360M-Instruct, and TinyLlama-1.1B-Chat) with greedy decoding. Of 420 graded responses, 271 (64.5%) failed their ground-truth oracle; pass rates ranged from 19% to 47%, string reversal was solved once in 90 attempts, and in 25 of 150 model–skeleton pairs a model answered some paraphrases correctly and others incorrectly. Without ground truth, the differential oracle flagged 98.8% of the failures with 73.9% precision but could not say which model was wrong; the metamorphic oracle was more precise (88.0%) and less complete (79.5%). Only one prompt received the same wrong answer from all three models. Coverage-guided expansion reached full grammar coverage in 27 of 30 seeds against 4 of 30 for random expansion, but only slightly increased the diversity of revealed failures. A manual check of 60 responses agreed with the automatic verdicts in 95% of cases, and every disagreement came from the strictness of the JSON oracle. The results suggest that verifiable grammar derivations make differential and metamorphic signals actionable, while oracle design, rather than input generation, limits what can be concluded.*

Keywords: *software testing; grammar-based fuzzing; differential testing; metamorphic testing; large language models.*

1. Introduction

Chat models built on large language models (LLMs) are now ordinary software components: they answer questions, transform text, and produce structured output that other programs parse. The Hugging Face Hub distributes hundreds of open chat models, many of them small enough to run on a laptop, and exposes them both as downloadable weights and through a hosted inference API. A developer who embeds such a model faces the same question as with any third-party component: on which inputs does it misbehave? Answering this question is hard for three reasons. Chat models rarely crash, so the crash oracle that drives most fuzzers reveals little. Their input language is natural language, which has no parser whose structure a fuzzer could exploit. And for most prompts there is no single correct output, which makes the test-oracle problem acute [Barr et al. 2015]. Hand-written examples, in turn, explore only a small and biased part of the input space.

Fuzzing research offers tools for each of these obstacles. Grammar-based fuzzing generates large numbers of structured, valid inputs from a grammar [Holler et al. 2012], and coverage of the grammar itself can guide generation when the internals of the system under test cannot be observed [Havrikov and Zeller 2019]. Differential testing replaces a missing specification by comparing several implementations on the same input [McKeeman 1998], and metamorphic testing replaces it by comparing a system's outputs on related inputs [Chen et al. 1998]. Neither comparison, however, establishes that an output is wrong: two LLMs may legitimately phrase the same answer differently, and a disagreement does not say which model erred. Existing LLM fuzzers mostly target jailbreaks, whose oracle is whether an attack succeeded [Yu et al. 2023], and metamorphic frameworks for LLMs compare each model only with itself [Hyun et al. 2024, Cho et al. 2025]. We did not find work that generates prompts from a grammar, guides generation by grammar coverage, and applies both a cross-model differential oracle and a cross-paraphrase metamorphic oracle to open chat models, nor work that measures how often these oracles are right when the true answer is known.

Research problem and objective. This paper studies whether grammar-based fuzzing combined with differential and metamorphic testing is an effective way to reveal functional failures in small open chat models, and what limits the conclusions it supports. We make the oracle problem tractable by design: our grammar generates *verifiable* prompts, whose expected answer follows from the values drawn during generation (arithmetic, letter counting, string reversal, JSON output, and verbatim echo of hard Unicode text), plus a category of free-form requests with no expected answer. Because the ground truth is known, we can measure what the differential and metamorphic oracles would have reported without it. Specifically, we aim to (i) quantify and characterize the failures the approach reveals, (ii) measure the recall and precision of the differential and metamorphic oracles against ground truth, (iii) evaluate whether coverage-guided expansion improves on random expansion, and (iv) estimate how often the automatic oracles misjudge a response.

This paper makes the following contributions:

1. `chatfuzz`, an open-source grammar-based, coverage-guided fuzzer for chat models that generates paraphrase families with built-in ground truth and supports local Hugging Face models and the Hugging Face Inference Providers API.

2. An empirical study on three small open chat models (Qwen2.5-0.5B-Instruct, SmolLM2-360M-Instruct, and TinyLlama-1.1B-Chat) that characterizes 271 oracle failures in 420 graded responses, including failures that only paraphrasing reveals.

3. A quantitative comparison of oracles: using ground truth as reference, we report how much of the failure set the differential and metamorphic oracles recover, how often they raise false alarms, and how rarely all models agree on the same wrong answer.

4. An evaluation of coverage guidance on grammar coverage and failure diversity, and a manual validation of the automatic oracles that locates their main source of error.

Section 2 reviews related work, Section 3 states the research questions, Sections 4 and 5 describe the method and the experimental setup, Section 6 reports the results, Section 7 discusses them, Section 8 covers threats to validity, and Section 9 concludes.

2. Background and Related Work

Our approach combines four lines of work that developed largely in separate communities: grammar-based fuzzing, differential testing, metamorphic testing, and the testing of LLMs. We review each line, then compare the approaches along the dimensions that determine what a test campaign can conclude: how inputs are generated, how valid they are, which oracle judges the output, and whether the result is reproducible.

2.1. Grammar-Based Fuzzing and Grammar Coverage

Random fuzzing produces inputs that most structured programs reject at the parser, so the code behind the parser is rarely exercised. Grammar-based fuzzers derive syntactically valid inputs from a grammar instead. LangFuzz, a black-box fuzzer driven by a context-free grammar and fragments of existing test programs, found 105 severe vulnerabilities in Mozilla's JavaScript interpreter in three months [Holler et al. 2012]. Validity has a price: a grammar only generates the structures its author anticipated. When the internals of the system cannot be observed, coverage of the input grammar itself can guide generation toward structures not yet tried. Havrikov and Zeller [2019] formalized k-path coverage of a grammar, and the `GrammarCoverageFuzzer` of *The Fuzzing Book* [Zeller et al. 2024] prefers expansions not yet used, which corresponds to 1-path coverage and is the direct basis of our generator. These fuzzers target formal languages, whose grammar is the specification of valid input. Natural language has no such grammar, so a prompt grammar does not describe the input space of a chat model; it describes a chosen family of requests. A prompt can be well formed by our grammar and still be ambiguous for a model, a distinction we return to in Section 4.

2.2. Differential Testing

Differential testing gives the same input to several systems that should behave alike and treats divergence as a failure [McKeeman 1998]. Its strength is that it needs no specification; its weakness is that a divergence is meaningful only if the systems are supposed to agree. Csmith addressed this for C compilers by generating only programs without

undefined behavior, so that every disagreement indicates a bug; it found more than 325 previously unknown bugs, and every compiler tested both crashed and silently generated wrong code [Yang et al. 2011]. DeepXplore brought the idea to deep learning, using several models with the same function as cross-referencing oracles that need no labels, and searching by gradient for inputs that maximize both disagreement and neuron coverage [Pei et al. 2017]. Chat LLMs share no formal specification and are trained differently, so a disagreement among them is a signal rather than a verdict, and a disagreement does not identify the faulty model. Work on hallucination detection shows the complementary risk: cross-checking a model against other models catches errors that self-consistency misses, but models can also be consistently wrong [Zhang et al. 2023], in which case the differential oracle is silent.

2.3. The Test-Oracle Problem and Metamorphic Testing

Deciding whether an output is correct is the test-oracle problem [Barr et al. 2015]. Barr et al. distinguish specified oracles, derived oracles such as metamorphic relations, and implicit oracles such as crashes; free-form chatbots are an extreme case because specified oracles are rarely available. Metamorphic testing [Chen et al. 1998] sidesteps the problem by checking relations between the outputs of related inputs. In NLP, CheckList generates tests from templates and checks invariance under meaning-preserving changes; in a user study, practitioners who used it created twice as many tests and found almost three times as many bugs [Ribeiro et al. 2020]. For LLMs, METAL generates hundreds of metamorphic relations from templates to test quality attributes such as robustness [Hyun et al. 2024]; Cho et al. [2025] collected 191 metamorphic relations for NLP tasks and ran about 560 thousand metamorphic tests on three LLMs; and MORTAR extends metamorphic testing to multi-turn dialogue systems, finding 51% more bugs per test case than the best single-turn method across eight systems [Guo et al. 2026]. All of these frameworks compare a model with itself. A violated relation shows inconsistency but not which output is wrong, and a satisfied relation can hide a consistently wrong answer.

2.4. Testing LLMs and Chat Models

Most LLM fuzzing targets security. GPTFuzzer mutates human-written jailbreak templates and reaches attack success rates above 90% against ChatGPT and Llama-2 [Yu et al. 2023]; its oracle is a classifier that judges whether the attack succeeded. Wang et al. [2024] review 102 studies on LLMs in software testing, nearly all of which use LLMs to test other software rather than test the LLMs themselves. NLP research, meanwhile, documents functional weaknesses that a test generator can target. Meaning-preserving formatting changes can shift accuracy by up to 76 points, and this sensitivity persists with larger models and instruction tuning [Sclar et al. 2024]. Models that know how their tokens are spelled still fail to manipulate text at the character level, for example to reverse or count letters [Edman et al. 2024]. These findings come from static benchmarks evaluated mostly on models of 7 B parameters or more.

Testing LLMs also differs from testing deterministic programs. Sampling makes outputs vary between runs, providers update hosted models without notice, and generation parameters such as temperature and the token limit change the output. A discrepancy can therefore stem from the decoding configuration or the deployment rather than from the

model, and a test campaign must either control these factors or repeat each test enough times to separate them. We control them: local models, pinned revisions, and greedy decoding make every reply reproducible on a given software stack.

2.5. Comparison and Positioning

Table 1 contrasts representative approaches. Grammar- and template-based generators (LangFuzz, CheckList, METAL) guarantee input validity but limit diversity to what their authors wrote; mutation-based generators (GPTFuzzer) explore more freely but can drift from the intended task. Oracles trade generality for decisiveness: differential and metamorphic oracles apply to any input but only flag suspicious behavior, whereas ground-truth oracles decide correctness but only exist for tasks whose answer can be computed. Cost and reproducibility follow from the system under test: DeepXplore needs gradients, hosted LLMs charge per request and change over time, and local models with greedy decoding are cheap to rerun and deterministic.

Table 1. Representative approaches compared with ours.

Approach	Target	Input generation	Oracle	Known answer
LangFuzz [2012]	JS engine	grammar + code fragments	crash	no
Csmith [2011]	C compilers	grammar without undefined behavior	differential	no
DeepXplore [2017]	DL models	gradient search	differential	no
CheckList [2020]	NLP models	templates	labels + invariance	partly
GPTFuzzer [2023]	LLMs	mutation of templates	attack classifier	no
METAL [2024]	LLMs	templates	metamorphic (self)	no
MORTAR [2026]	dialogue systems	dialogue perturbation	metamorphic (self)	no
This work	chat LLMs	grammar + coverage guidance	ground truth + differential + metamorphic	yes

To our knowledge, no prior work combines a context-free prompt grammar with grammar-coverage guidance, a cross-model differential oracle, and a cross-paraphrase metamorphic oracle for open chat models. Each ingredient has a canonical precedent: the `GrammarCoverageFuzzer` for generation, Csmith and DeepXplore for the differential oracle, and CheckList for the invariance oracle. What the combination adds is that each grammar derivation carries its own expected answer. This lets us settle disagreements between models and, more importantly for this study, measure how often the differential and metamorphic oracles would have been right without that answer. The price is scope: the approach only covers tasks whose answer can be computed from the generated values.

3. Research Questions and Objectives

Our objective is to assess whether grammar-based fuzzing combined with differential and metamorphic oracles is an effective way to reveal functional failures in small open chat models, and where the approach falls short. We address four research questions. Table 2 lists, for each one, the metrics and the data that answer it.

RQ1 (Effectiveness). How many failures, and of which kinds, does grammar-based fuzzing reveal in small open chat models from the Hugging Face Hub, and at what cost? *Motivation:* a technique is only worth adopting if it finds failures that matter at a cost a developer can afford.

RQ2 (Oracles). What do the differential (cross-model) and metamorphic (cross-paraphrase) oracles add beyond oracles that rely on a known expected answer? *Motivation:* for most real prompts no expected answer exists, so the practical value of the approach depends on these two oracles. Because our prompts do have an expected answer, we can measure their recall and precision instead of assuming them.

RQ3 (Guidance). Does coverage-guided grammar expansion improve grammar coverage and the diversity of revealed failures compared with uniform random expansion under the same budget? *Motivation:* coverage guidance is the main design choice of the generator and has a cost in implementation complexity.

RQ4 (Oracle reliability). How often do the automatic oracles misjudge a response, as measured on a manually inspected sample? *Motivation:* every number in RQ1–RQ3 depends on heuristic oracles; RQ4 bounds their error.

Table 2. Research questions, metrics, and data sources.

RQ	Metrics	Data
RQ1	failures by type and category; pass rate per model with 95% Wilson intervals; paired McNemar tests between models; replies cut at the token limit; latency	main campaign
RQ2	recall and precision of the differential and metamorphic flags against the ground-truth verdict; prompts on which all models give the same wrong answer	main campaign, verifiable categories
RQ3	grammar coverage after n prompts; prompts to full coverage; failures, revealing rules, and distinct failure signatures	Ablations 1 and 2
RQ4	agreement and Cohen's κ between automatic verdict and manual judgment	stratified sample of 60 responses

We state no directional hypotheses for RQ1, RQ2, and RQ4, which are descriptive. For RQ3, the hypothesis implied by prior work [Zeller et al. 2024, Havrikov and Zeller 2019] is that coverage guidance reaches full grammar coverage with fewer prompts; it would be refuted if random expansion reached full coverage as often and as fast. Whether higher grammar coverage also yields more diverse failures is an open question that Ablation 2 explores without enough seeds for a significance test.

4. Methodology

Our tool, `chatfuzz`, has four stages (Figure 1): a grammar-based prompt generator, chatbot clients, oracles, and an analysis stage that applies the differential and metamorphic relations. The code, the grammar, and 19 unit tests are in the project repository.

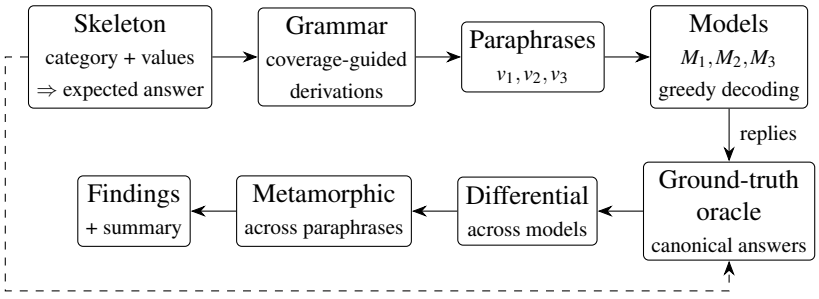


Figure 1. The `chatfuzz` pipeline. The expected answer travels with the test case (dashed arrow) and is never shown to the models.

4.1. Test Case Generation

Each test case starts from a *skeleton*: a task category and randomly drawn values, such as the operands of an arithmetic expression. Because the values are stored, every test case knows its expected answer. A context-free grammar then derives the wording of the request, filling nonterminals such as `<a>` and `<word>` from the skeleton's values. Deriving the same skeleton several times yields *paraphrases* that must receive the same answer. The grammar has 16 nonterminals with 41 expansions in total, including one start symbol per category (Table 3). For example, `<arith>` has three top-level templates, the operation word depends on the drawn operation (+ or *plus*; $\star$, *times*, or *multiplied by*), and the closing instruction has three variants (*Answer with only the number*, *Reply with just the final number*, *Give only the numeric result*).

Table 3. Task categories, example prompts generated by the grammar, and ground-truth oracles.

Category	Example prompt	Oracle
`arith`	*How much is 795772 minus 43450? Reply with just the final number.*	last integer in the reply equals the result
`count`	*How many times does the letter 'r' appear in the word 'strawberry'?*	last integer equals the true count
`reverse`	*Write 'W2Cv' backwards. Reply with the result only.*	reply, stripped of quotes and final period, equals the reversed string
`json`	*Return a JSON object with the keys "age", "score" and "email". Reply with JSON only.*	reply (optionally inside a Markdown code fence) parses as a JSON object with the requested keys
`echo`	*Repeat exactly the following text, without any change:* followed by emoji, right-to-left or invisible characters	payload appears verbatim in the reply
`edge`	*Translate to Portuguese:* followed by a hard Unicode payload	no runtime error and a non-empty reply

Operands mix one-digit, three-digit, six-digit, and negative numbers, and the operation is addition, subtraction, or multiplication. Words for `count` come from a list of ten words with repeated letters (e.g., *strawberry*, *mississippi*, *referrer*); the letter is drawn from the word with probability 0.85 and from outside it otherwise, so that the expected count can be zero. Strings for `reverse` are 4 to 12 random letters and digits, which avoids words a model could have memorized. JSON requests ask for two to four keys out of eight. The `echo` and `edge` categories draw from 12 payloads: combining emoji (*woman technologist*, a rocket, and a flag), Arabic and Japanese text, zero-width spaces, ligatures, accented words, mathematical symbols, a tab-separated string, a two-line string, a JSON snippet, HTML markup, and a 300-character repetition. The generator applies no automatic capitalization or normalization, because doing so altered the payloads (it turned the ligature *fi* into *FI*).

The grammar guarantees that every prompt is well formed and states a task whose answer follows from the skeleton. It does not guarantee that the prompt is unambiguous to a model, nor that a model's tokenizer represents the payload faithfully; these properties are what the tests probe. The `edge` category is deliberately different: its requests (summarize, translate, or repeat a hard payload, or the bare payload) have no expected answer, so only the implicit oracle of errors and empty replies applies, complemented by manual inspection.

4.2. Coverage-Guided Expansion and Paraphrases

When a nonterminal has several expansions, the generator chooses an expansion not yet used with probability b (`coverage_bias`, 0.8 by default) and a uniformly random one otherwise. This follows the grammar coverage fuzzer of Zeller et al. [2024] and corresponds to 1-path grammar coverage [Havrikov and Zeller 2019]. Grammar coverage is the fraction of the 41 (nonterminal, expansion) pairs used so far. For each skeleton, the generator draws up to k derivations and keeps only those whose words differ from the derivations already kept, ignoring whitespace; a variant that differs only in a line break tests nothing. If the grammar cannot produce k distinct wordings for a category, fewer paraphrases are kept. Generation is seeded, so the same seed and parameters reproduce the same prompts, and each test case records the grammar expansions used to derive it.

4.3. Chatbot Clients

Models are addressed as `local:<model>` or `api:<model>`. Local models are downloaded from the Hugging Face Hub and run with the `transformers` library, using the model's own chat template, a single user turn without a system prompt, and greedy decoding, so that each prompt has a single deterministic reply. The API client sends OpenAI-compatible chat completion requests to the Hugging Face Inference Providers router and retries on rate limits, server errors, and timeouts. Both clients record runtime errors, timeouts, and malformed replies as data rather than raising them, and flag a reply that stops at the token limit as truncated.

4.4. Oracles and Analysis

The analysis applies three kinds of oracle, matching the taxonomy of Barr et al. [2015]:

1. **Ground-truth (specified) oracles** check each reply against the expected answer of its category (Table 3) and extract a *canonical answer*: an integer, a normalized string, the sorted key list of a JSON object, or a Boolean for `echo`. The numeric oracle takes the last integer in the reply because models often restate the expression before the result; it accepts thousands separators and the Unicode minus sign.

2. **Differential oracle.** The same prompt is sent to every model. Different canonical answers produce a *differential disagreement*, which also names the models that are wrong when the expected answer is known; a model that fails at runtime while others reply produces a *differential error*. When all models give the same wrong answer, the analysis reports a *consensus-wrong* prompt, a case that a purely differential oracle cannot detect [Zhang et al. 2023].

3. **Metamorphic (derived) oracle.** For each model, the paraphrases of one skeleton must yield the same canonical answer; otherwise the tool reports a *metamorphic inconsistency*.

Greedy decoding removes sampling as a source of discrepancy, so a disagreement between paraphrases or models cannot be explained by chance on a fixed software stack. Two other sources of spurious discrepancy remain, and the design handles them explicitly: replies cut at the token limit are flagged and counted separately, and differences in surface form are reduced by comparing canonical answers rather than raw text. For `echo`, the canonical answer is the oracle's verdict itself, so we exclude `echo` from the consensus-wrong check and from the oracle comparison of RQ2. The tool reports, per model, the pass rate by category, metamorphic consistency, pairwise disagreement between models, and the number of runtime errors and truncated replies.

5. Experimental Setup

Systems under test. We selected three small instruction-tuned chat models from different families, so that differences between them are not artifacts of a shared training pipeline (Table 4). All three are distributed through the Hugging Face Hub and ran locally, with revisions pinned, greedy decoding, and at most 256 new tokens. We implemented the client for the hosted Hugging Face Inference Providers API but did not use it in the experiments, because no API token was available in the execution environment; the systems under test are therefore the models as served by `transformers`, not a hosted API.

Table 4. Models under test.

Model	Parameters	Revision	Reference
`Qwen/Qwen2.5-0.5B-Instruct`	0.5 B	`7ae5576`	Qwen Team [2024]
`HuggingFaceTB/SmolLM2-360M-Instruct`	360 M	`a10cc15`	Ben Allal et al. [2025]
`TinyLlama/TinyLlama-1.1B-Chat-v1.0`	1.1 B	`fe8a4ea`	Zhang et al. [2024]

Environment. A Linux cloud container with 4 CPU cores and 15 GB of RAM, no GPU, Python 3.13, PyTorch 2.14.1, and `transformers` 5.21.0. Latencies are wall-clock times per reply on this machine, with one request at a time.

Main campaign (RQ1, RQ2, RQ4). With grammar seed 0 and $b = 0.8$, the generator produced 60 skeletons, 10 per category, each derived as up to 3 paraphrases. The `echo` templates admit only two distinct wordings, so the campaign has 170 prompts (161 distinct strings, because some skeletons repeat) and 510 replies. The 140 prompts of the five verifiable categories yield 420 graded replies; the 90 `edge` replies are inspected manually. Every model received the same prompts.

Ablation 1: grammar coverage without models (RQ3). For 30 grammar seeds, we generated 120 prompts with uniform random expansion ($b = 0$) and with coverage-guided expansion ($b = 0.8$), and recorded grammar coverage after each prompt.

Ablation 2: failure diversity with models (RQ3). For grammar seeds 0, 1, and 2 and both values of b, we ran 30 skeletons with one wording each on the two faster models, Qwen2.5 and SmolLM2 (60 replies per condition, 360 in total). We compare the number of failures, the number of distinct grammar expansions that appear in failing prompts (*revealing rules*), and the number of distinct failure signatures (model, category, oracle verdict, and top-level template).

Oracle validation (RQ4). We drew a stratified random sample of four graded replies per model and verifiable category (60 replies, sampling seed 0). For each reply we recorded two judgments: whether the reply gives the correct answer, tolerating extra text that does not change it (*content*), and whether it also follows the requested format, e.g. "only the number" (*strict*). We compare both with the automatic verdict. In addition, we labeled all 90 `edge` replies as benign, refusal, hallucinated or wrong, degenerate repetition, or chat-template leakage. The judgments were made with the assistance of an AI model and are pending independent review by the authors (see the AI use disclosure).

Analysis. We report proportions with 95% Wilson score intervals and compare models on the same prompts with the exact McNemar test. For RQ2 we restrict the analysis to `arith`, `count`, `reverse`, and `json`, whose canonical answers are independent of the oracle verdict (360 replies). A reply is *flagged by the differential oracle* if its canonical answer, or the absence of one, differs from that of at least one other model on the same prompt, and *flagged by the metamorphic oracle* if the model did not give the same canonical answer to all paraphrases of the skeleton. Recall is the share of ground-truth failures that are flagged; precision is the share of flagged replies that are failures. All statistics are computed by `experiments/paper_stats.py` from the saved replies.

6. Results

The main campaign took 53.9 minutes of model time for 510 replies. No reply produced a runtime error or was empty, so all findings below come from the ground-truth, differential, and metamorphic oracles. The 27 model–prompt pairs in which the same prompt string occurred twice received identical replies, which confirms that the setup is deterministic.

6.1. RQ1: Effectiveness

Of the 420 graded replies, 271 (64.5%) failed their ground-truth oracle: 215 gave a wrong answer, 30 contained no parsable JSON object, 22 did not reproduce the payload, and 4 contained no number. Table 5 and Figure 2 show the pass rates. SmolLM2-360M

and Qwen2.5-0.5B passed 47.1% and 40.0% of their prompts, a difference that is not significant on the same 140 prompts (McNemar, 21 vs. 11 discordant prompts, $p = 0.11$). TinyLlama-1.1B, the largest model, passed 19.3%, significantly fewer than both ($p < 0.001$ for each comparison).

Table 5. Pass rate (%) by model and category in the main campaign, with 95% Wilson intervals for the overall rate. n is the number of graded prompts per model.

Model	`arith` n=30	`count` n=30	`reverse` n=30	`json` n=30	`echo` n=20	Overall n=140	Truncated of 170
Qwen2.5-0.5B	33.3	36.7	0.0	100.0	25.0	40.0 [32.3, 48.3]	3
SmolLM2-360M	26.7	30.0	3.3	100.0	90.0	47.1 [39.1, 55.4]	9
TinyLlama-1.1B	10.0	30.0	0.0	0.0	75.0	19.3 [13.6, 26.6]	25

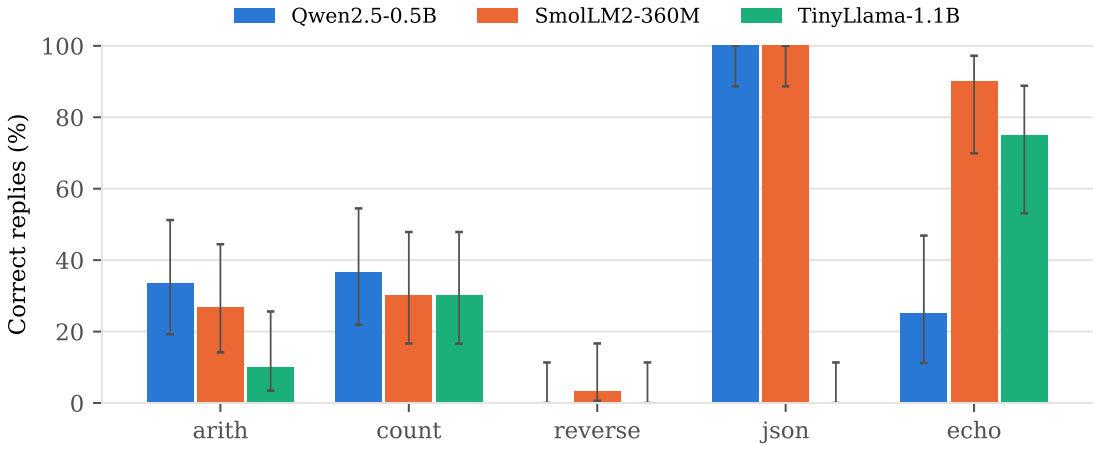


Figure 2. Share of correct replies per category and model; error bars are 95% Wilson intervals.

The failures fall into five kinds, which differ sharply between categories and models:

- **Character-level manipulation.** String reversal was solved once in 90 attempts. In 21 of the 89 failures the model returned the input unchanged (11 by TinyLlama, 8 by SmolLM2), and in most others it produced a scrambled string of similar length. Letter counting succeeded in 30% to 37% of prompts; for the two *a*'s of *onomatopoeia*, the replies ranged from 0 to 35. These results extend to models below 1.1 B parameters the character-level weakness reported for larger models [Edman et al. 2024].

- **Arithmetic.** Accuracy depended on operand size: 44.4% of the replies were correct when both operands had fewer than four digits, against 9.3% when at least one had six. Most wrong results (77%) were within a factor of ten of the correct value (Qwen answered 627,000,000 for $569 \times 110834 = 63{,}064{,}546$), but some replies repeated an operand or concatenated the operands.

- **Format violations.** TinyLlama never returned a bare JSON object: 25 of its 30 replies contained a valid object with the requested keys inside a Markdown code block surrounded by prose, and 5 wrote a Python function instead. Qwen and SmolLM2 returned valid JSON in all 60 replies. Across categories, TinyLlama often explained or wrote code instead of answering, for example a Python loop to count letters that never stated the count.
- **Payload corruption.** In the `echo` category Qwen reproduced only 5 of 20 payloads. It replaced the *woman technologist* emoji by *man technologist* and appended an unrelated Chinese sentence, answered an Arabic greeting in Korean, and rephrased Japanese and Arabic text. SmolLM2 copied 18 of 20 payloads and TinyLlama 15. The only `echo` prompt that all models failed was the tab-separated string, whose tabs were turned into hyphens, spaces, or a description of the TSV format.
- **Paraphrase sensitivity.** In 25 of 150 model–skeleton pairs (Qwen 12, TinyLlama 7, SmolLM2 6), the model answered some paraphrases correctly and others incorrectly. Table 6 shows a case in which changing only the closing instruction changed Qwen's count.

Table 6. A metamorphic failure revealed by grammar paraphrases (expected answers: 3 for *r*, 2 for *t*).

Prompt	Qwen	SmolLM2	TinyLlama
How many times does the letter 'r' appear in the word 'strawberry'? Give only the numeric result.	**3**	1	–
In the word "strawberry", how many 'r' characters are there? Give only the numeric result.	1	4	2
Count the occurrences of the letter 'r' in 'strawberry'. Give only the numeric result.	10	1	1
In the word "strawberry", how many 't' characters are there? Give only the numeric result.	1	4	**2**
In the word "strawberry", how many 't' characters are there? Answer with only the number.	**2**	4	**2**

The `edge` category, which has no expected answer, produced no runtime error or empty reply, so its automatic oracle found nothing. Manual inspection of its 90 replies found 33 benign replies, 29 hallucinated or wrong ones (e.g. a translation of *naïve café résumé* as "requerente de café náutico", or a summary of the repeated string *abab...* as a sentence about the universe), 13 refusals (7 by Qwen and 6 by SmolLM2, often to harmless HTML markup), 12 degenerate repetition loops that ran to the token limit, and 3 replies by TinyLlama that leaked its chat-template marker `<|user|>`. Of the 37 truncated replies, 24 were `edge` replies.

Cost. Mean latency per reply was 2.8 s for Qwen, 2.7 s for SmolLM2, and 13.6 s for TinyLlama, whose verbose replies took 71% of the campaign's model time. The campaign revealed about five ground-truth failures per minute of CPU time.

Answer to RQ1. Grammar-based fuzzing revealed failures in 64.5% of the graded replies, across all three models and all five verifiable categories, at a cost of under an hour

of CPU time. The failures concentrate on character-level tasks and large operands; JSON output separates the models more than any other category; and paraphrasing alone exposes inconsistencies in up to a quarter of a model's skeletons.

6.2. RQ2: What the Differential and Metamorphic Oracles Add

Table 7 compares, on the 360 replies of the four categories whose canonical answers are independent of the verdict, what the differential and metamorphic oracles would have flagged without ground truth. The differential oracle flagged 246 of the 249 failures (98.8%), but 87 of its 333 flags (26.1%) fell on correct replies. These false alarms arise because a disagreement marks every model, including the one that was right. Ground truth resolved these disagreements: among the 111 prompts with disagreeing answers, 34 had two correct models, 19 had exactly one, and 58 (52%) had none, so picking the majority answer would often have been wrong as well. The metamorphic oracle was more precise (88.0%) but less complete (79.5%): it missed failures in which a model gave the same wrong answer to every paraphrase. Together, the two oracles missed one failure.

Table 7. Oracles without ground truth, evaluated against the ground-truth verdict (360 replies, 249 failures; categories `arith`, `count`, `reverse`, `json`).

Oracle	Flagged	Failures flagged	Recall (%)	Precision (%)
Differential (across models)	333	246	98.8	73.9
Metamorphic (across paraphrases)	225	198	79.5	88.0
Either	344	248	99.6	72.1

Precision varied by category. It was highest for `reverse` (98.9%), where nearly every reply was wrong, and lowest for `json` (33.3%): every TinyLlama format failure made all three models disagree, so the two correct replies on each prompt were flagged as well. Only one prompt received the same wrong answer from all three models, which a purely differential oracle cannot see: all three counted one *a* in *onomatopoeia*, which has two. Sixty of the 140 graded prompts were failed by every model, but with different answers.

Answer to RQ2. With three weak and diverse models, the differential oracle recovered almost all failures, but about one flag in four was a false alarm and it could not identify the faulty model, which in half of the disagreements was every model. The metamorphic oracle traded recall for precision. Ground truth was what turned both signals into verdicts; shared wrong answers, the blind spot of differential testing, were rare (1 prompt).

6.3. RQ3: Coverage Guidance

Figure 3 and Table 8 show Ablation 1. Both strategies covered about a third of the 41 grammar expansions after 6 prompts; afterwards coverage guidance pulled ahead, reaching 80% after 24 prompts against 71% for random expansion. Within 120 prompts, coverage guidance reached full coverage in 27 of 30 seeds (median 73 prompts), random expansion in only 4 of 30 (median 94 among those).

In Ablation 2, coverage guidance did not change how many failures were found

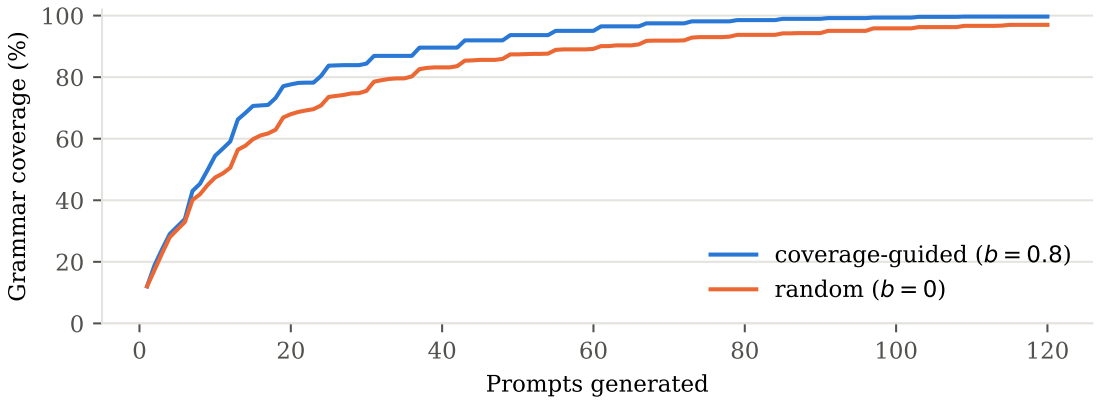


Figure 3. Mean grammar coverage over 30 seeds as prompts are generated (Ablation 1).

Table 8. Ablation results. Ablation 1: grammar coverage (mean of 30 seeds, 120 prompts each). Ablation 2: two models, 30 prompts per seed, mean over seeds 0–2 (values per seed in parentheses).

Metric	Random ($b = 0$)	Guided ($b = 0.8$)
Coverage after 6 / 12 / 24 / 48 prompts (%)	33 / 51 / 71 / 86	34 / 59 / 80 / 92
Seeds reaching 100% coverage within 120 prompts	4 / 30	27 / 30
Median prompts to 100% coverage	94	73
Failures per seed (of 60 replies)	30.0 (29, 28, 33)	30.3 (31, 28, 32)
Revealing rules	21.7 (22, 18, 25)	23.7 (21, 22, 28)
Distinct failure signatures	17.0 (18, 16, 17)	18.7 (18, 18, 20)

(30.0 vs. 30.3 per seed), because the failure rate is driven by the categories, which both strategies sample equally. It produced slightly more diverse failures: on average 2 more grammar expansions appeared in failing prompts and 1.7 more distinct failure signatures were observed. The guided condition had at least as many signatures as the random one in all three seeds and more revealing rules in two, but with three seeds and differences of one or two units these results are indicative only.

Answer to RQ3. Coverage guidance clearly improves grammar coverage, reaching all expansions within 120 prompts in 27 of 30 seeds against 4 of 30. Its effect on failure diversity is in the same direction but small and not statistically established at our scale; it does not increase the number of failures.

6.4. RQ4: Oracle Reliability

On the stratified sample of 60 replies, the automatic verdict agreed with the content judgment in 57 cases (95.0%, 95% CI [86.3, 98.3]; Cohen's $\kappa = 0.89$). The oracle never accepted a reply the judgment rejected; all three disagreements were TinyLlama replies with a valid JSON object wrapped in prose, which the JSON oracle rejects and a reader accepts. Agreement was 100% for `arith`, `count`, `reverse`, and `echo`, and 75% for `json`. Against the strict judgment, which also requires the requested format, agreement was again 57 of 60, with the opposite error: the oracle accepted a correct result preceded by the restated expression, a correctly copied payload followed by a repetition loop, and

a Japanese payload with an added full stop. Only 17 of the 60 replies (28%) met the strict standard.

The JSON oracle is thus the main source of oracle error, and its effect on Table 5 is large. A lenient variant that accepts a JSON object embedded in text raises TinyLlama's `json` pass rate from 0% to 83% (25 of 30) and leaves the other models unchanged. We also observed one verdict that was right for the wrong reason: for a reply consisting of Python code, the numeric oracle extracted the 1 of `r_count += 1`.

Answer to RQ4. The heuristic oracles agreed with manual judgment on 95% of the sample. Their errors are systematic rather than random: the numeric and echo oracles are lenient about extra text, the JSON oracle is strict, and the choice between leniency and strictness changes one model's result in one category from 0% to 83%.

7. Discussion

Input generation was not the bottleneck; oracles were. Every verifiable category revealed failures in every model, and coverage guidance made the grammar exercise all of its expansions with fewer prompts. Finding failures in models of this size is easy. What the campaign can conclude, however, depends on decisions made in the oracles: whether a JSON object inside prose counts (0% or 83% for TinyLlama), whether the last integer of a reply is its answer, whether a copied payload followed by a repetition loop is a copy. Our oracles are lenient in some categories and strict in another, and RQ4 shows that this asymmetry, not random noise, accounts for their disagreement with manual judgment. A test campaign on chat models should therefore report the oracle's definition of correctness as part of its results, and ideally report results under both a content and a format criterion, because instruction following and task competence are different properties: only 28% of the sampled replies met both.

Differential testing finds almost everything and decides almost nothing. With three weak models from different families, nearly every failure produced a disagreement, so the differential oracle had near-perfect recall. This is the setting in which differential testing is most effective at detection and least informative about blame: in half of the disagreements no model was right, so neither a majority vote nor trust in the strongest model would have produced the correct answer. Csmith made disagreements decisive by restricting generation to programs with defined behavior [Yang et al. 2011]; our analogue is restricting generation to tasks with a computable answer. The two strategies share the same trade-off: decisiveness is bought by narrowing the input space. Consensus-wrong prompts, the blind spot that motivated combining models with ground truth [Zhang et al. 2023], were rare in our data (one prompt), probably because the models fail in different ways. With stronger and more similar models we would expect more shared errors and lower differential recall.

Paraphrases expose a failure class that the other oracles do not name. The metamorphic oracle was the more precise of the two ground-truth-free oracles, and its signal has a direct interpretation: the model's answer depends on wording that should not matter. A change of the closing instruction alone changed Qwen's answer (Table 6), which mirrors the formatting sensitivity reported for larger models [Sclar et al. 2024]. For a developer, a metamorphic inconsistency is also a reproducible bug report: two prompts,

same task, different answers. Its blind spot is the consistently wrong model, which is the case where ground truth or a second model is needed.

Model size did not predict robustness. TinyLlama, the largest model tested (1.1 B parameters), had the lowest pass rate, mostly because it ignored output instructions: it explained, wrote code, or wrapped JSON in prose. SmolLM2, the smallest (360 M), copied hard Unicode payloads better than Qwen, which rewrote or translated them. Differences in instruction tuning appear to matter more than size at this scale. These comparisons hold for our prompts and decoding configuration only, and we do not claim a ranking of the models in general.

Practical implications. For practitioners who embed small chat models, the results suggest three checks that a grammar-based campaign can automate cheaply: whether the model follows output-format instructions, whether its answer is stable under paraphrase, and whether it preserves non-ASCII text it is asked to copy. For researchers, the results suggest that the effectiveness of LLM fuzzing should be evaluated with known-answer tasks before trusting differential or metamorphic signals on open-ended ones, because recall and precision of these oracles can be measured only there.

Limits of the technique. The approach covers only tasks whose answer can be computed from the generated values. The `edge` category shows what lies outside that scope: hallucinated translations, unwarranted refusals, degenerate repetition, and template leakage occurred in 57 of 90 replies, and none was detected automatically. Grammar coverage, finally, measures the variety of request templates, not the variety of model behaviors; RQ3 shows that guidance improves the former, while its effect on the latter remains small at our scale.

8. Threats to Validity and Limitations

Construct validity. The ground-truth oracles rely on heuristics: the numeric oracle takes the last integer in the reply, the reversal oracle strips quotes and a final period, the echo oracle accepts the payload anywhere in the reply, and the JSON oracle rejects objects embedded in prose. RQ4 shows that these choices cause systematic errors, and the JSON choice alone changes one model's pass rate in one category from 0% to 83%; we report the lenient variant alongside the strict one. The differential oracle compares canonical answers, so two equivalent replies with different surface forms may count as a disagreement, and the precision we report for it depends on the canonicalization. The `edge` category only checks for runtime errors and empty replies; our classification of its replies is manual and coarse.

Internal validity. Greedy decoding makes each local model deterministic for a given software stack, and repeated prompts received identical replies, but results may change with other library versions, hardware, or chat templates. Replies cut at the 256-token limit are flagged and reported (37 of 510, 10 of them graded failures), not discarded; a larger limit could turn some of these failures into passes. The manual judgments for RQ4 and the `edge` labels were made with the assistance of an AI model and have not yet been independently checked by the authors; a second human rater would allow an inter-rater agreement measure. The sample of 60 replies gives a wide confidence interval (86–98% agreement).

External validity. We tested three small models (360 M to 1.1 B parameters) running locally, not the hosted Hugging Face API, which we implemented but could not use without an access token. Larger models, hosted deployments with sampling, and other providers may behave very differently; in particular, stronger models would likely fail less often and share more errors, which would lower the recall of the differential oracle. The prompts are in English, single-turn, and cover six task categories; they do not represent open-ended conversation, and multi-turn effects [Guo et al. 2026] are outside our scope.

Conclusion validity. The main campaign uses one grammar seed and ten skeletons per category, so per-category rates have wide intervals (Table 5), and some sub-cases, such as letters absent from the word, were not drawn at all. Ablation 2 uses three seeds, so its differences are indicative rather than statistically established. Ablation 1, which needs no models, uses 30 seeds.

Naturalness of generated prompts. Grammar-derived prompts are well formed but templated, and the `echo` grammar admits only two wordings. Real users phrase requests more freely, so our metamorphic results bound paraphrase sensitivity from below for the wordings we wrote, not for natural variation in general.

9. Conclusion and Future Work

We combined grammar-based, coverage-guided fuzzing with differential testing across models and metamorphic testing across paraphrases, and evaluated the combination on three small open chat models from the Hugging Face Hub. Because every grammar derivation carries its expected answer, we could measure not only how many failures the approach reveals but also how reliable its oracles are. The approach revealed failures in 64.5% of the graded replies, concentrated on character-level tasks, large operands, output-format instructions, and Unicode payloads, and exposed answers that change with wording alone. Without ground truth, the differential oracle would have flagged almost all failures but could not have said which model was wrong, while the metamorphic oracle was more precise and missed consistently wrong answers. Coverage guidance reached full grammar coverage far more reliably than random expansion. The main limitation we found lies in the oracles rather than in input generation: their definition of a correct reply changed one model's result in one category from 0% to 83%.

Future work includes running the campaign against hosted models through the Hugging Face API, with sampling and repeated queries to separate non-determinism from inconsistency; testing larger and more similar models, where shared wrong answers should be more frequent; extending the grammar with multi-turn dialogues and with tasks whose answers can be checked by a program, such as code or unit conversions; using an independent human rater for the oracle validation; and measuring behavioral coverage of the model, not only coverage of the grammar.

AI Use Disclosure

During the preparation of this work, the authors used Claude (Anthropic), through Claude Code, to implement the fuzzing tool and its unit tests, run the experiments, search and summarize related work, judge the oracle-validation sample and label the `edge` replies, and draft this paper. These judgments have not yet been independently reviewed by the

authors. All numerical results come from the runs described in Section 5 and can be reproduced with the scripts in the project repository. The authors are responsible for reviewing the content; bibliographic entries marked as unverified must be checked against the original sources before submission.

References

Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., and Yoo, S. (2015). The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525.

Ben Allal, L. et al. (2025). SmolLM2: When smol goes big. arXiv:2502.02737.

Chen, T. Y., Cheung, S. C., and Yiu, S. M. (1998). Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology. Available as arXiv:2002.12543.

Cho, S., Ruberto, S., and Terragni, V. (2025). Metamorphic testing of large language models for natural language processing. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 174–186.

Edman, L., Schmid, H., and Fraser, A. (2024). CUTE: Measuring LLMs' understanding of their tokens. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Guo, G., Aleti, A., Neelofar, N., Tantithamthavorn, C., Qi, Y., and Chen, T. Y. (2026). MORTAR: Multi-turn metamorphic testing for LLM-based dialogue systems. *IEEE Transactions on Software Engineering*.

Havrikov, N. and Zeller, A. (2019). Systematically covering input structure. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 189–199.

Holler, C., Herzig, K., and Zeller, A. (2012). Fuzzing with code fragments. In *21st USENIX Security Symposium*, pages 445–458.

Hyun, S., Guo, M., and Babar, M. A. (2024). METAL: Metamorphic testing framework for analyzing large-language model qualities. In *IEEE Conference on Software Testing, Verification and Validation (ICST)*.

McKeeman, W. M. (1998). Differential testing for software. *Digital Technical Journal*, 10(1):100–107. Partially verified.

Pei, K., Cao, Y., Yang, J., and Jana, S. (2017). DeepXplore: Automated whitebox testing of deep learning systems. In *ACM Symposium on Operating Systems Principles (SOSP)*.

Qwen Team (2024). Qwen2.5 technical report. arXiv:2412.15115.

Ribeiro, M. T., Wu, T., Guestrin, C., and Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4902–4912.

Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. (2024). Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations*

(ICLR). arXiv:2310.11324.

Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., and Wang, Q. (2024). Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*.

Yang, X., Chen, Y., Eide, E., and Regehr, J. (2011). Finding and understanding bugs in C compilers. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.

Yu, J., Lin, X., Yu, Z., and Xing, X. (2023). GPTFUZZER: Red teaming large language models with auto-generated jailbreak prompts. arXiv:2309.10253.

Zeller, A., Gopinath, R., Böhme, M., Fraser, G., and Holler, C. (2024). Grammar coverage. In The Fuzzing Book. CISPA Helmholtz Center for Information Security. `https://www.fuzzingbook.org/html/GrammarCoverageFuzzer.html`. Accessed: 2026-10-29.

Zhang, J., Li, Z., Das, K., Malin, B., and Kumar, S. (2023). SAC3: Reliable hallucination detection in black-box language models via semantic-aware cross-check consistency. In *Findings of the Association for Computational Linguistics: EMNLP*.

Zhang, P., Zeng, G., Wang, T., and Lu, W. (2024). TinyLlama: An open-source small language model. arXiv:2401.02385.